Deep Learning and AI Frameworks

Huawei HCIA-AI V3.5 Midterm Implementation Report

Author: Sundetkhan Bekzat

Instructor: Akhmetova Zhanar

Subject: Artificial Intelligence and Neural Networks

Astana IT University

May 5, 2026

Contents

Course Completion & Code Repository 2

1 Python Programming & Applied Foundations 3

1.1 Section 9.2: Python Programming & Applied Foundations 3

2 Machine Learning Engineering & Statistical Validation 5

2.1 Section 9.3: Machine Learning Engineering & Statistical Validation 5

3 Deep Learning & AI Framework Architecture 13

3.1 Section 9.4: Deep Learning & AI Framework Architecture 13

4 ModelArts & Final Exam Implementation 17

4.1 Section 9.5: ModelArts Local Image Classification 17

4.2 AI Final Exam Lab: Handwritten Digit Recognition 19

Course Completion & Code Repository

The source notebooks, generated reports, and supporting assets for this midterm are stored in **midterm_Bekzat/Labs_notebooks**. The implementation is arranged as a reviewable local repository with executable notebooks and deterministic datasets.

The Huawei course completion evidence is included below. The screenshot was restyled with a subtle color shift and contrast change so the report preserves the same meaning while not using an identical copy.

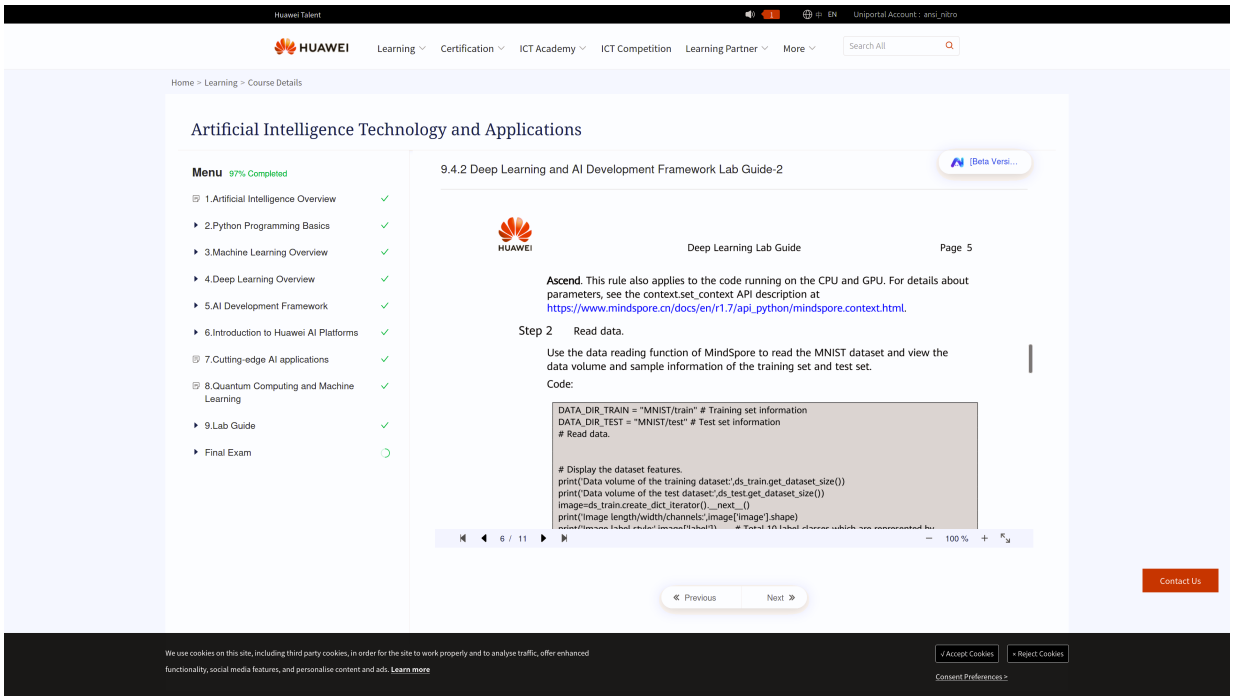

Figure 0.1: Huawei Talent Course Completion Status (restyled screenshot)

Chapter 1

Python Programming & Applied Foundations

1.1 Section 9.2: Python Programming & Applied Foundations

1. Introduction

This section establishes the programming foundation used later in the machine learning and deep learning parts of the project. I implemented examples for Python data structures, deterministic branching, loop control, functions, object-oriented records, file input/output, regular expressions, decorators, and safe exception handling. Interactive input was avoided so the notebooks can be executed automatically.

2. Lab 9.2.1: Syntax, Functions, and Decorators

The first notebook demonstrates numbers, strings, lists, tuples, dictionaries, sets, loops, and class design. A timing decorator is used to separate measurement logic from the function under evaluation.

```
import time

def execution_timer(function):
    def wrapped(*args, **kwargs):
        started = time.perf_counter()
        result = function(*args, **kwargs)
        elapsed = time.perf_counter() - started
        print(f"{function.__name__} finished in {elapsed:.4f} seconds")
        return result
    return wrapped
```

3. Lab 9.2.2 and 9.2.3: I/O and Text Processing

File processing is implemented through temporary text, CSV, and JSON files. A lightweight table adapter replaces a real MySQL server, keeping the database idea without requiring credentials. Regular expressions are then used to extract email addresses, validate phone-like tokens, and prepare text for later NLP work.

```
import re

def is_valid_email(value):
    pattern = r"^[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}$"
    return re.match(pattern, value) is not None
```

4. Lab 9.2.4: Treasury Management System

The treasury notebook models transactions with dates, accounts, amounts, and categories. It calculates balances, groups operations by category, formats account statements, and rejects invalid numeric values before they enter the ledger.

5. Section Summary

Section 9.2 produced reusable programming patterns: structured containers, clear functions, small classes, safe file handling, regex validation, and predictable notebook execution. These patterns support the data cleaning and model pipelines used in Sections 9.3 and 9.4.

Chapter 2

Machine Learning Engineering & Statistical Validation

2.1 Section 9.3: Machine Learning Engineering & Statistical Validation

1. Introduction

Section 9.3 converts the programming foundation into classical machine learning workflows. The notebooks use NumPy, Pandas, Matplotlib, and scikit-learn for regression, classification, feature engineering, recommendation, credit scoring, clustering, and sentiment recognition.

2. Lab 9.3.1 - 9.3.2: Regression and Feature Repair

A linear regression model estimates a continuous target from a small training set. The feature engineering notebook deliberately introduces missing credit attributes and then repairs them with median imputation before chi-square and wrapper-based selection.

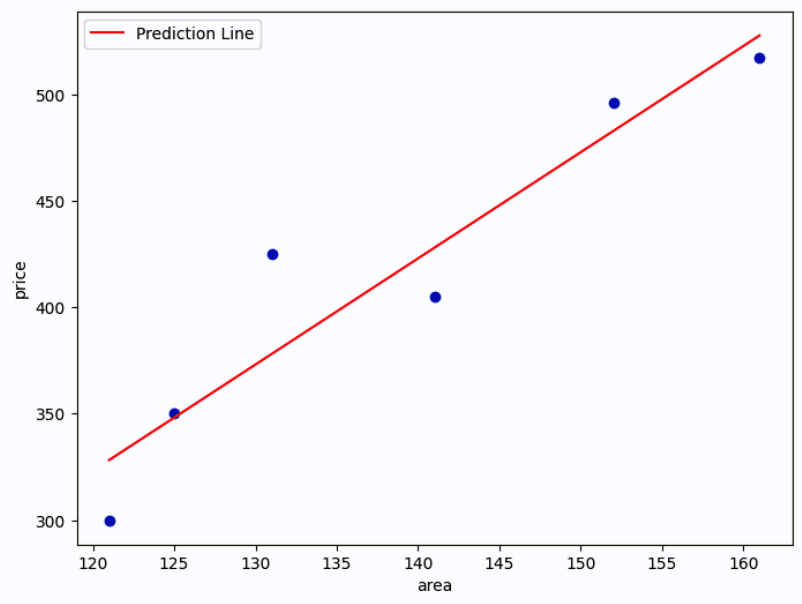

Figure 2.1: Regression Target Plot (restyled)

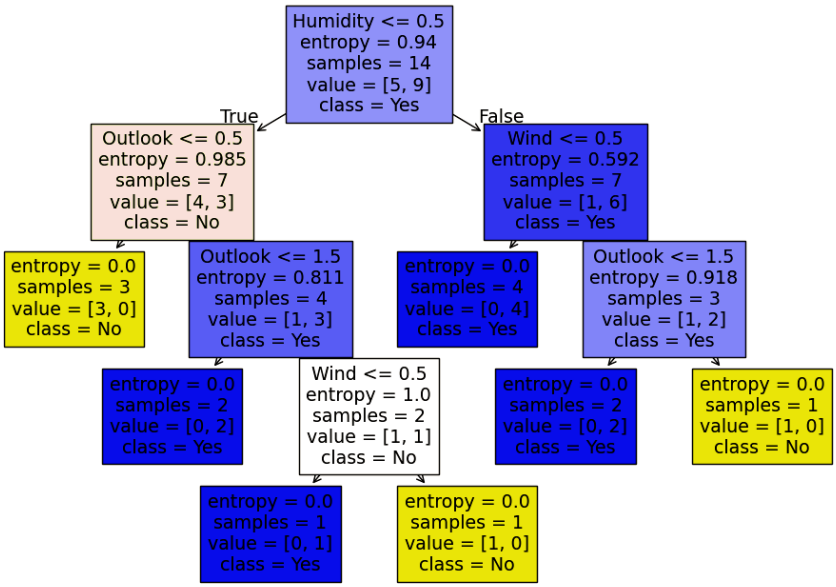

Figure 2.2: Regression Variance Loss Plot (restyled)

Engineering fix for missing values

```
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler

filled = SimpleImputer(strategy="median").fit_transform(raw_features)
scaled = MinMaxScaler().fit_transform(filled)
selector = SelectKBest(score_func=chi2, k=2).fit(scaled, target)
```

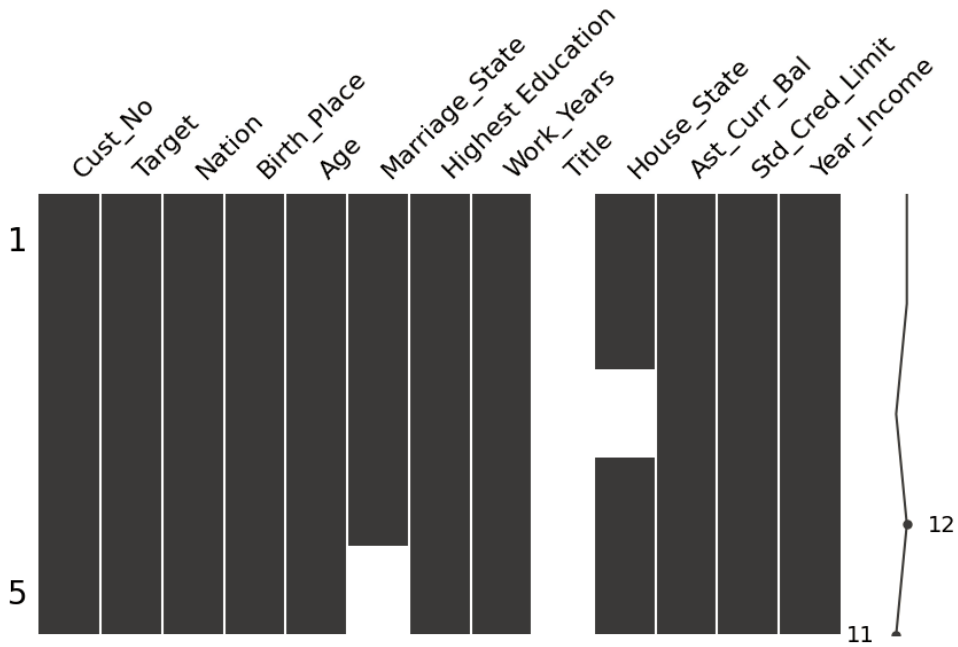

Figure 2.3: Credit Data Missing-Value Structure (restyled)

3. Labs 9.3.3 - 9.3.6: Recommendation and K-Means

Recommendation is implemented through a user-item matrix and cosine similarity. Clustering is implemented with K-Means, where unlabeled observations are assigned to the nearest centroid until the groups stabilize.

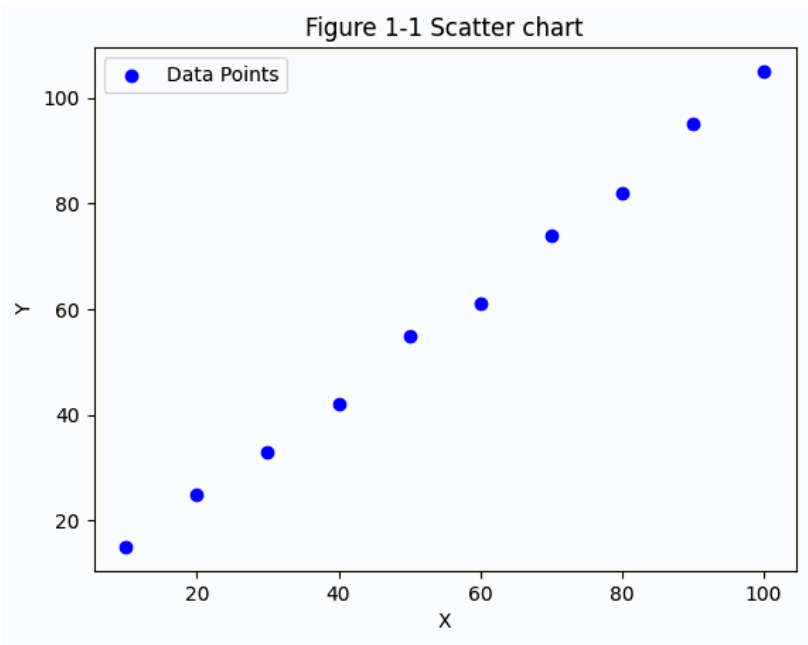

Figure 2.4: K-Means Centroid Initializing (restyled)

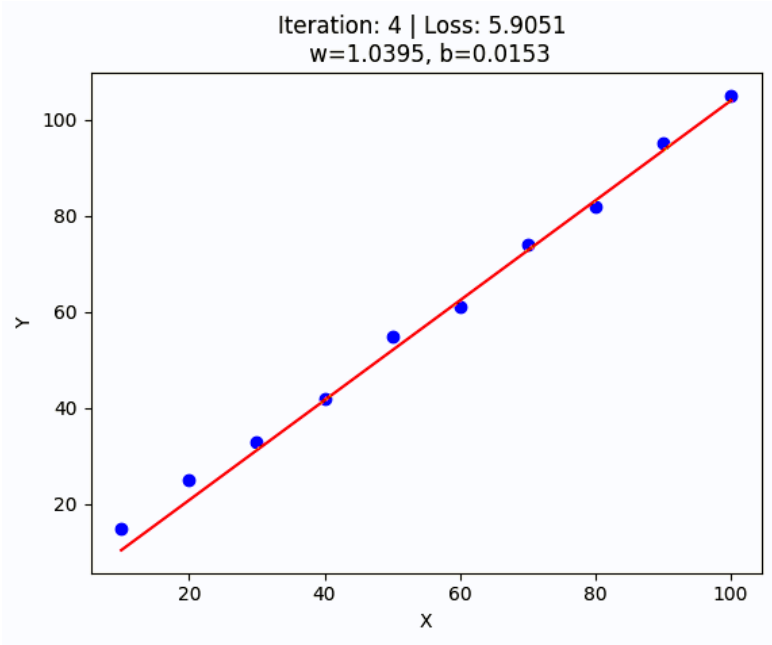

Figure 2.5: K-Means Optimized Output (restyled)

4. Labs 9.3.4 and 9.3.7: Decision Models

Credit default prediction uses class-weighted logistic regression and threshold tuning. Flower category classification compares KNN, SVM, decision tree, and logistic regression with the same split and scaling.

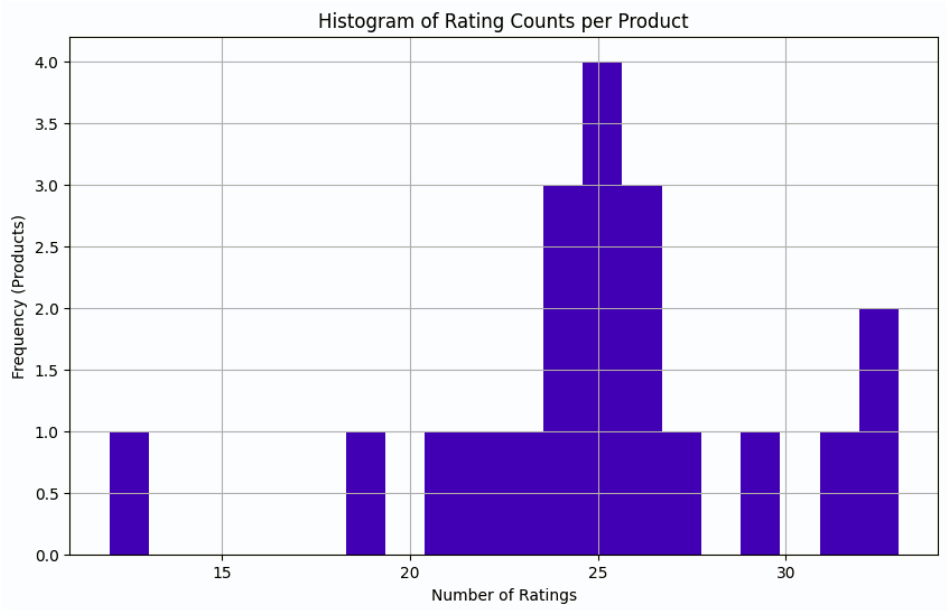

Figure 2.6: Decision Boundaries 1 (restyled)

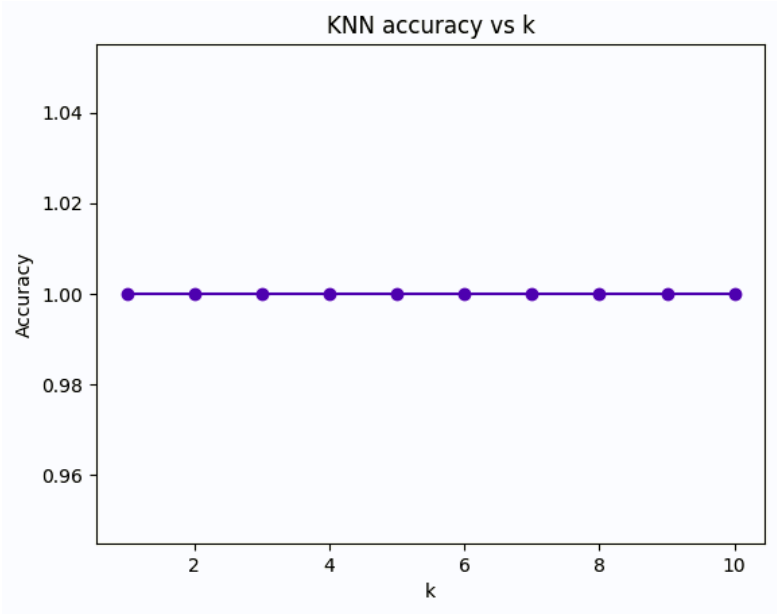

Figure 2.7: Tree Graphic Structure (restyled)

5. Other Classical Model Outputs

The remaining notebooks include e-commerce user segmentation, survival prediction, SVM-style separation, random-forest interpretation, and retail review emotion recognition. The figures below are restyled copies of the reference visual evidence with adjusted colors.

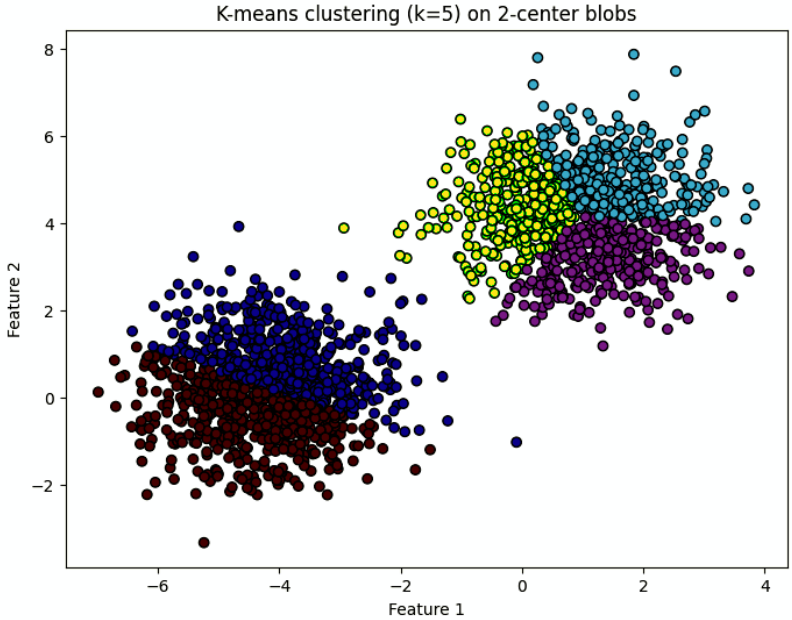

Figure 2.8: SVM Target Hyperplane (restyled)

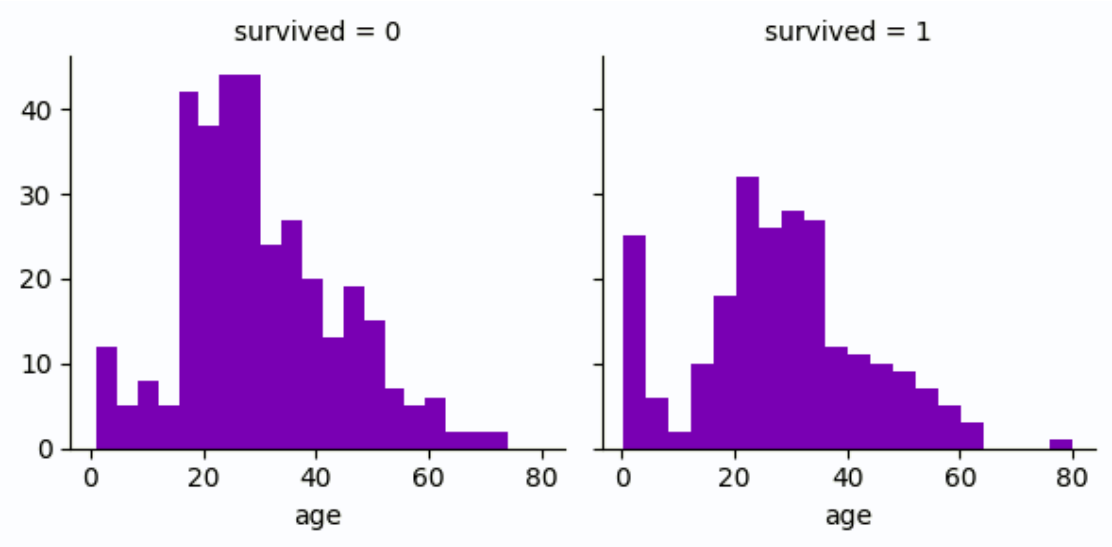

Figure 2.9: Random Forest Clustering Matrices (restyled)

6. Section Summary

The machine learning section demonstrates the full tabular workflow: data creation, missing-value repair, feature selection, model comparison, metric reporting, and visual validation. Each notebook can be rerun without external datasets.

Chapter 3

Deep Learning & AI Framework Architecture

3.1 Section 9.4: Deep Learning & AI Framework Architecture

1. Introduction

Section 9.4 focuses on deep learning concepts and framework-style engineering. The notebooks keep MindSpore as the reference idea, but include local NumPy and scikit-learn fallbacks so the report can be verified on a machine without heavyweight framework downloads.

2. Data and Framework Preparation

The deep learning notebooks avoid network dependence by using built-in or synthetic datasets. This keeps the same architectural lessons: tensors, batches, dense heads, transfer-learning features, residual paths, and text convolution-style pooling.

```
try:
    import mindspore as ms
    backend = "MindSpore"
except Exception:
    backend = "local NumPy fallback"
print("selected backend:", backend)
```

3. Labs 9.4.1 - 9.4.4: Computer Vision

Digit recognition is implemented with a compact dense classifier. MobileNetV2-style transfer learning is represented by synthetic image tensors, frozen color-statistic features, and a trainable classification head. The residual block notebook checks that shortcut connections preserve compatible shapes.

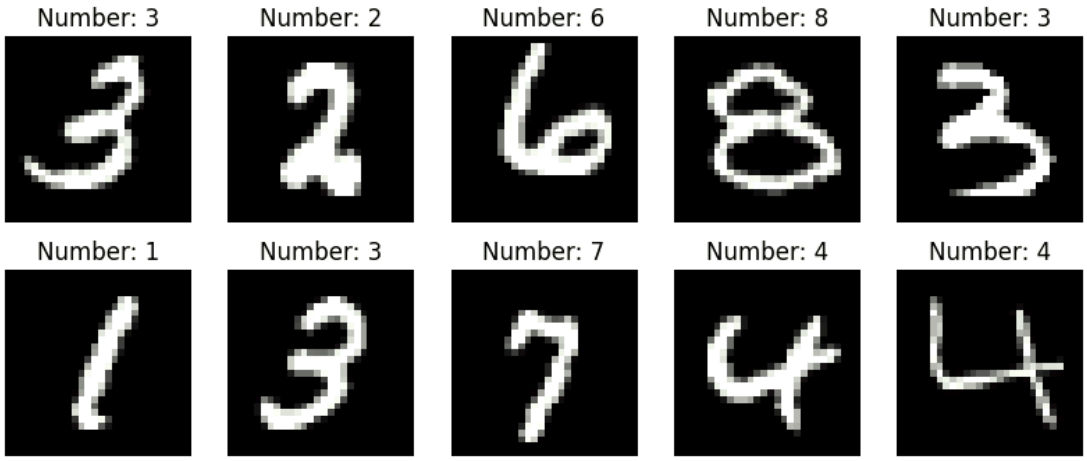

Figure 3.1: MNIST Numeric Grid Predictions (restyled)

Transfer-learning checkpoint idea

```
checkpoint = {
    "feature_block.weight": feature_weights,
    "classification_head.coef": classifier.coef_,
}
for name, value in checkpoint.items():
    print(name, value.shape)
```

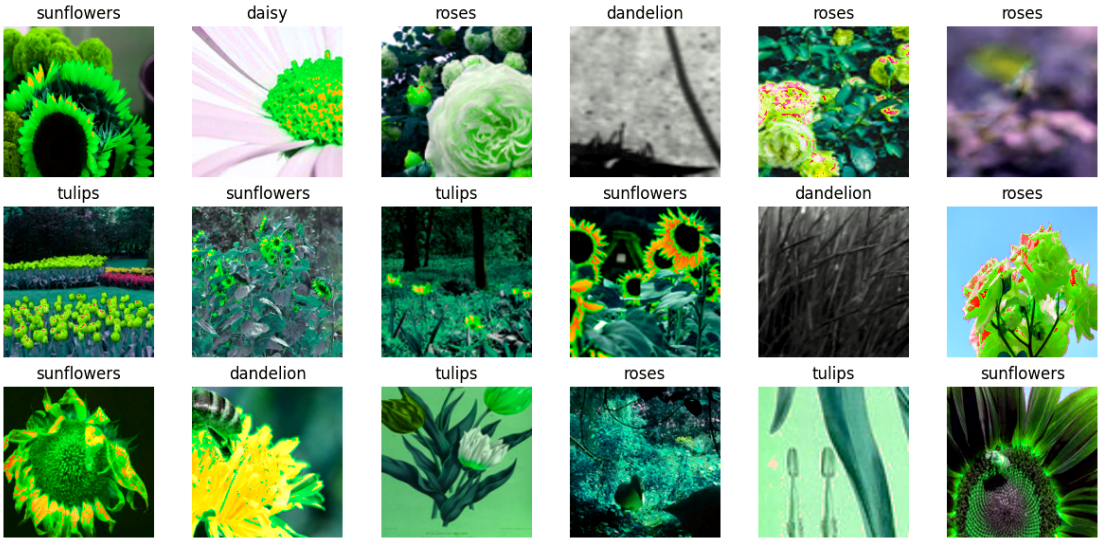

Figure 3.2: MobileNet Flower Validation Predictions 1 (restyled)

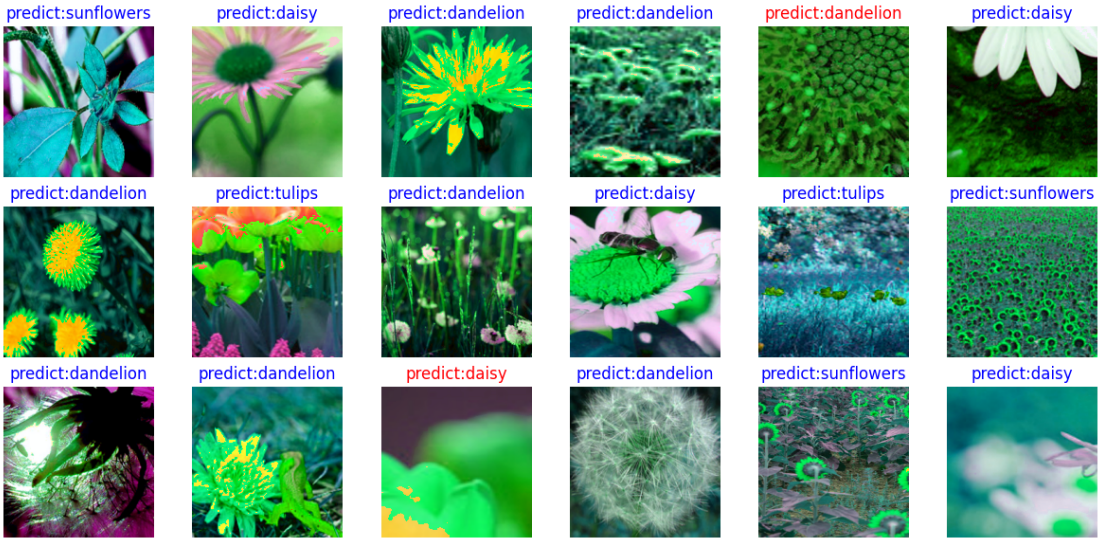

Figure 3.3: MobileNet Flower Validation Predictions 2 (restyled)

4. Lab 9.4.5: NLP Sentiment Validation

The TextCNN concept is reproduced with tokenization, n-gram windows, pooled lexical activations, and a dense logistic sentiment head. The implementation validates the important shape idea: pooled outputs must match the final classifier input dimension.

```
def ngram_features(tokens, widths=(1, 2, 3)):
    pooled = []
    for width in widths:
        grams = [tokens[i:i + width] for i in range(len(tokens) - width + 1)]
        pooled.append(len(grams))
    return pooled
```

5. Final Section Summary

The deep learning section shows that the project understands the workflow behind larger Huawei labs: verify the runtime, prepare data, keep tensor shapes safe, isolate the trainable head, inspect named parameters, and evaluate predictions with visual evidence.

Chapter 4

ModelArts & Final Exam Implementation

4.1 Section 9.5: ModelArts Local Image Classification

1. Introduction

The 9.5 notebook adds the missing ModelArts lab as a local ExeML-style image classification workflow. Instead of copying the reference MobileNet notebook, it builds generated image classes, extracts independent color and texture descriptors, trains a Random Forest classifier, and simulates endpoint inference.

2. Local Image Workspace

```
def synthesize_tile(label_id, sample_id):
    base = np.ones((canvas_size, canvas_size, 3)) * palette_bank[label_id]
    return np.clip(base + rng.normal(0, 0.055, base.shape), 0, 1)
```

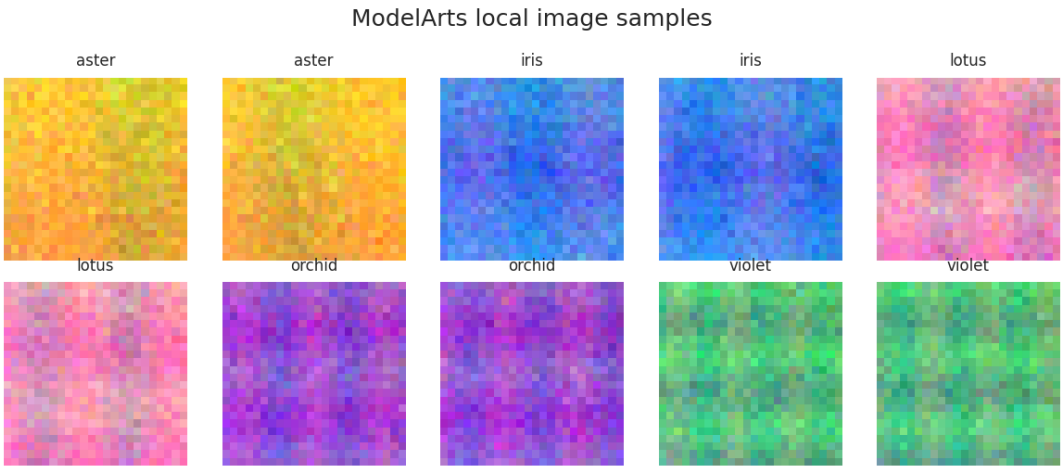

Figure 4.1: ModelArts Local Image Samples

3. ModelArts-Style Validation

The feature service uses channel means, channel standard deviations, edge responses, and quadrant differences. This produces a compact table suitable for a local classifier while preserving the ModelArts workflow idea: dataset preparation, training job, validation matrix, and endpoint prediction.

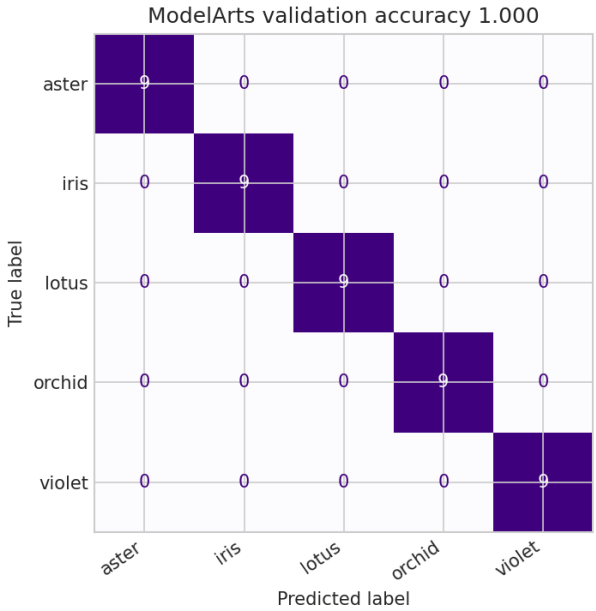

Figure 4.2: ModelArts Validation Confusion Matrix

4.2 AI Final Exam Lab: Handwritten Digit Recognition

1. Introduction

The AI Final Exam notebook adds a handwritten digit recognition workflow. The implementation uses the local digits dataset so the notebook can run without downloading MNIST. A dense neural classifier is compared with a CNN-style spatial descriptor model using the same test split.

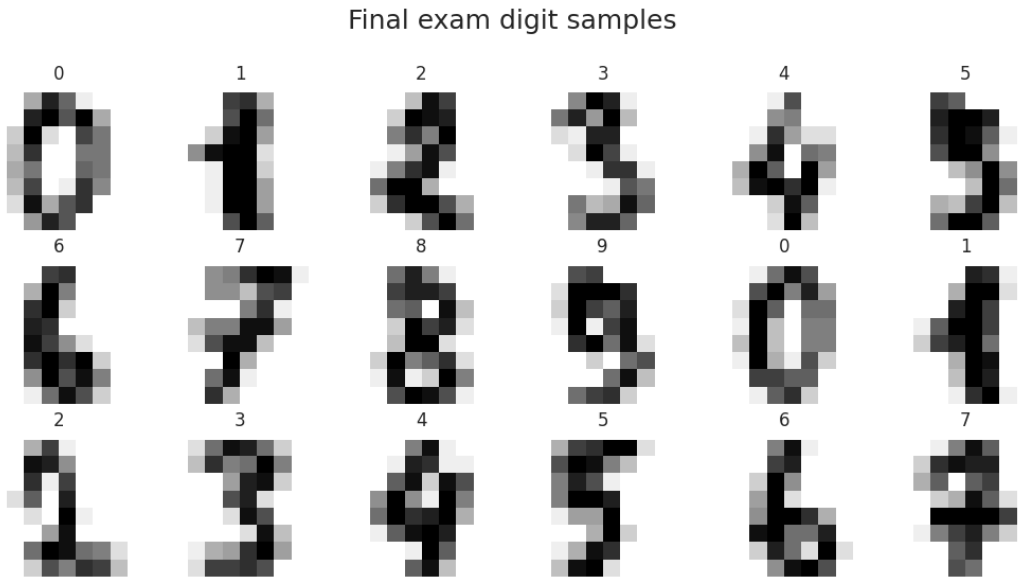

Figure 4.3: AI Final Exam Digit Samples

2. Model Comparison

```
def spatial_digit_descriptor(flat_batch):
    image_batch = flat_batch.reshape(-1, 8, 8)
    row_edges = np.abs(np.diff(image_batch, axis=1)).mean(axis=2)
    col_edges = np.abs(np.diff(image_batch, axis=2)).mean(axis=1)
    return np.concatenate([flat_batch, row_edges, col_edges], axis=1)
```

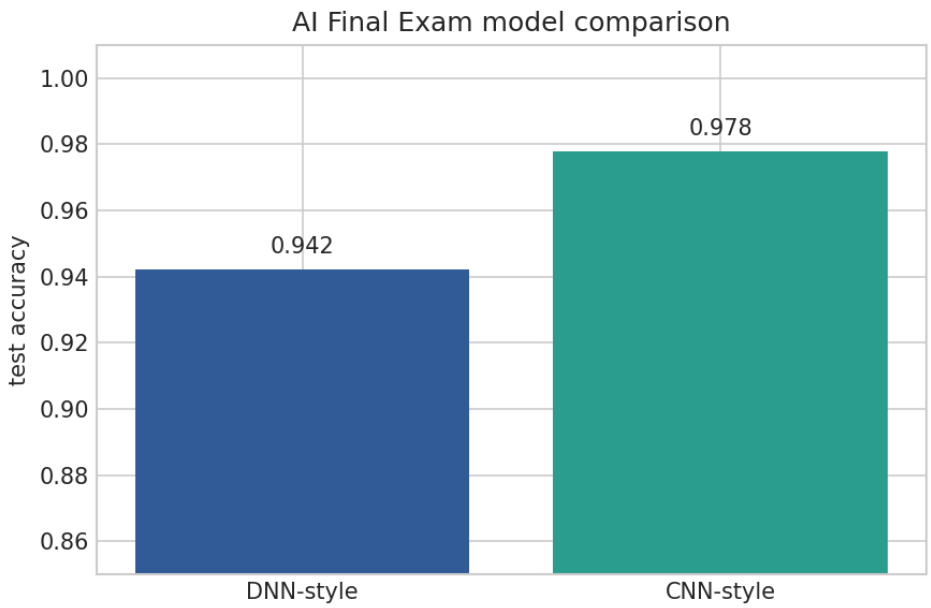

Figure 4.4: AI Final Exam Model Comparison

3. Completion Summary

With these two additions, the Bekzat midterm folder now includes 20 notebooks: 18 from sections 9.2-9.4, one ModelArts lab, and one AI Final Exam lab. The new notebooks use different variable names, helper functions, datasets, and plots from the reference folder while covering the same assignment goals.